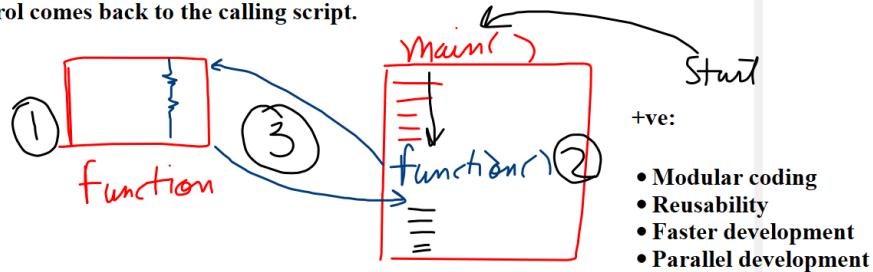


Your Function

1 Code written away from the actual script

2 Called when needed from the script

3 When call control gets transferred to the function, the function gets executed and control comes back to the calling script.



We can write a function before the main(preferred) or after the main part(needs extra coding called prototyping).

#include<>

//global area function can be here

int main()

{

././prototype

.

.

}

//after main area.....

//If a function is defined over here, the compiler needs function prototyping, which is a kind of prerequisite: telling the compiler that there is a function of so-and-so type coming after you.

define a function:Function body or function definition

syntax:

```
<return type> <name/identifier>(parameters)
{
    //code
}
```

//Prototyping is the first line of the function definition with a semicolon.

```
<return type> <name/identifier>(parameters);
```

It will introduce the main part and the compiler, indicating that there is a function of so-and-so type coming ahead.

A function has three parts:

- 1. Definition:body**
- 2. Call:activation**
- 3. Optional prototype:intro to main iff function is after main()**

1. What is a Function?

A **function** is a block of code designed to perform a **specific task**.

Instead of writing the same code again and again, we write it once inside a function and **call it whenever required**.

Real-life example

Think about a calculator.

You may have:

- add() → addition
- subtract() → subtraction
- multiply() → multiplication
- divide() → division

Each function performs one specific task.

Basic Structure

```
return_type function_name(parameters)
{
    // statements
}
```

Example:

```
void display()
{
```

```
        cout << "Hello";  
    }
```

Calling the function:

```
display();
```

Output:

Hello

2. Why Do We Need Functions?

Functions provide:

1. Code Reusability

Write once, use many times.

2. Modularity

Large programs can be divided into smaller logical units.

3. Easy Debugging

If something goes wrong, we can check a particular function.

4. Easy Maintenance

Changes can be made in one place.

5. Better Readability

Instead of:

```
// 50 lines of code
```

we can write:

```
calculateSalary();
```

The function name tells us what the code does.

3. Three Important Parts of a Function

A function generally has:

Function Declaration

↓

Function Definition

↓

Function Call

4. Function Declaration / Prototype

A **function declaration** tells the compiler:

"A function with this name and these parameters exists."

Syntax

```
return_type function_name(parameter_list);
```

Example:

```
int add(int, int);
```

or

```
int add(int a, int b);
```

Both are valid.

5. Function Definition

The actual implementation of the function is called its **definition**.

```
int add(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

Here:

- int → return type
- add → function name
- a, b → parameters
- return a + b → returned value

6. Function Call

A function is executed when it is called.

```
int result = add(10, 20);
```

Output:

30

Just to show some message or some copyright information

(Most of the time this can be used when you want to see a result temporarily not permanently.)

Generally return certain values from the system, like date, time, rate of conversion, etc.

Most of the industry-level solutions would require this. Even in an interview we tend to write this type of function because the actual script is pre-written. The student is supposed to complete only this kind of function.

```
cout<<"\nStart in the main part. ";
cout<<"\nSome code, some logic. ";
my_function();
```

```

my_function2("amar");
cout<<"\n\t\tInfo:"<<copyright();
cout<<"\n\t\tAmar Panchal initials:"<<initials("amar","panchal");
cout<<"\nEnd in main part";\
return 0;
}

```

-A function can call another function, called nesting of functions.

One function will be called by the user. That function internally calls another function.

example: You place an order on Swiggy or Zomato that internally places the order to the restaurant.

```

#include <iostream>
using namespace std;

```

```

int square(int n)
{
    return n * n;
}

int calculate(int n)
{
    return square(n);
}

int main()
{
    cout << calculate(5);
    return 0;
}

```

The diagram illustrates the execution flow of the provided code. Red arrows and numbers show the flow from `main` to `calculate(5)` and then to `square(5)`. A red arrow points from the `return` statement in `square` back to the `return square(n);` line in `calculate`, with the value `25` written in red. Green arrows and numbers show the flow from `main` to `calculate(5)` and then back to the `cout` statement in `main`, with the value `25` written in green. The value `5` is written in green next to the argument in `calculate(5)` and in red next to the argument in `square(5)`.

Function overloading. :

Multiple functions having exactly one identifier /name:

In this case the compiler can differentiate who is called on the basis of three options:

1. Type of parameters
2. Number of parameters
3. Both

```

#include <iostream>
using namespace std;

```

```

void add(int no1,int no2)
{
    cout<<"\nAdding integers :"<<no1<<"+"<<no2<<"="<<(no1+no2);
}
void add(float no1,float no2)
{
    cout<<"\nAdding float :"<<no1<<"+"<<no2<<"="<<(no1+no2);
}
void add(int no1,int no2,int no3)
{
    cout<<"\nAdding 3 integers :"<<no1<<"+"<<no2<<"+"<<no3<<"="<<(no1+no2+no3);
}
int main()
{
    add(10,20);
    add(12.40f,11.60f);
    add(12,23,34);
    add(int(10),45.02);//forced int given

    return 0;
}

```

Default Arguments

C++ allows us to provide default values to parameters.

```
int add(int a, int b = 10)
```

```

{
    return a + b;
}

```

Now:

```
cout << add(5);
```

Output:

15

Because:

a = 5

b = 10

We can also provide both:

```
add(5, 20);
```

Output:

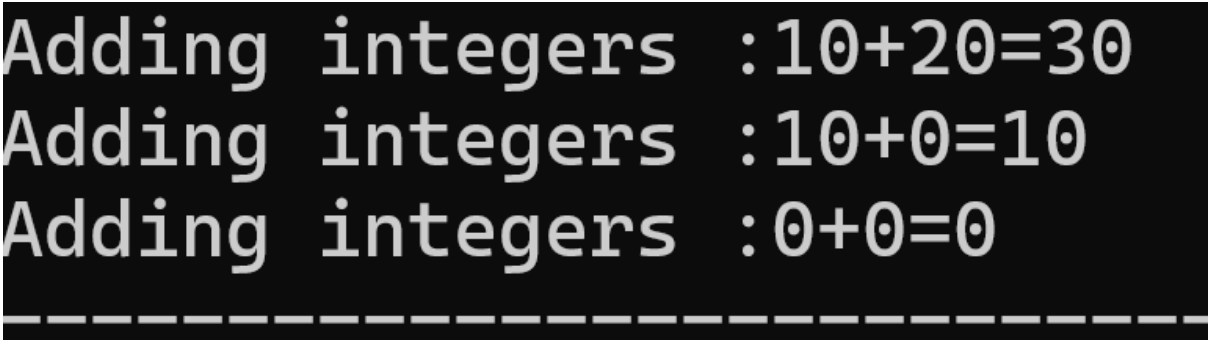
25

```
#include <iostream>
using namespace std;
//Default value parameter
//A parameter which will only be used if the user does not provide
//Values are given directly in the function definition and activated only if the user fails to
provide.
```

```
void add(int no1=0,int no2=0)
{
    cout<<"\nAdding integers : "<<no1<<"+"<<no2<<"="<<(no1+no2);
}
```

```
int main()
{
    add(10,20);
    add(10);
    add();

    return 0;
}
```



```
Adding integers :10+20=30
Adding integers :10+0=10
Adding integers :0+0=0
-----
```

Recursion function or recursive function

Recursion simply means reoccurrence.

A recursive function is a function which calls itself again and again.

+ve:

It makes code look smaller hence more readable.

-ve:

Recursion makes the program run slow.

Recursion takes more memory and recursion is complex to understand.

```
int factorial(int no)
```

```
{
```

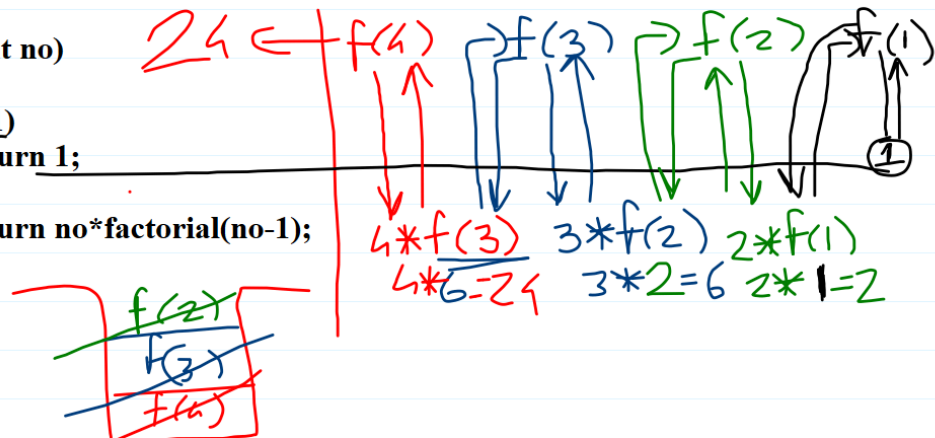
```
    if (no==1)
```

```
        return 1;
```

```
    else
```

```
        return no*factorial(no-1);
```

```
}
```



/*

Start

Accept two numbers no1 and no2.

Find the remainder when no1 is divided by no2.

Replace no1 with no2.

Replace no2 with the remainder.

Repeat steps 3–5 until the second number becomes zero.

The first number is the GCD.

Return the GCD.

Stop.

Lambda function, also known as an anonymous function

Functions which are used for a short time and then never referred to

Also known as throwaway functions because they are used for a short period of time

A normal function gets loaded into memory and then executed.

Lambda functions, on the other hand, are treated as temporary variables. The moment somebody stops using them they will be thrown away.

```
auto <var>=[ ](parameters)
    {
        //code
    };
auto add=[ ](int no1,int no2)
    {
        return(no1+no2);
    }
```

```

    };

    add(11,22)

*/

#include <iostream>
using namespace std;

int main()
{
    auto add=[](int no1,int no2)
    {
        return(no1+no2);
    };
    cout<<"Addition is:"<<add(11,22);
    return 0;
}

```

Lambda Functions

A lambda is an **anonymous function**.

Basic syntax:

```
[](parameters)
```

```
{
    // code

```

```
};
```

Example:

```
auto add = [](int a, int b)
```

```
{
    return a + b;

```

```
};
```

```
cout << add(10, 20);
```

Output:

30

Lambdas are especially useful with STL algorithms.

Example:

```

sort(v.begin(), v.end(), [](int a, int b)
{
    return a > b;
});

```

This sorts in descending order.

```

#include <iostream>
using namespace std;

```

```

/*
pointer to function:
Using a pointer concept can call a function.
As a parameter you can pass a function to another function.

```

Like a normal pointer this is also a pointer but would require prototyping with a parameter data type.

```

int add(int no1,int no2)
{
    return (no1+no2);
}

```

```

return type (*pointer) (data type,data type);
int (*p)(int,int);//Created pointer, along with its return type, its identifier, and its parameter
type.
p=add
//call
cout<<"Addition is:"<<p(10,20);

```

```

*/
int add(int no1,int no2)
{
    return (no1+no2);
}
int main()
{
    int (*p)(int,int);
    p=add;
    cout<<"Addition is:"<<p(10,20);

```

```
    return 0;
}
```

Function Pointer

A **function pointer** stores the address of a function.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Function pointer:

```
int (*ptr)(int, int);
```

Assign:

```
ptr = add;
```

Call:

```
cout << ptr(10, 20);
```

Output:

```
30
```

Think

Normal pointer:

```
int *p;
```

Function pointer:

```
int (*p)(int, int);
```

The brackets are extremely important.

Why Function Pointers?

They are useful for:

- callbacks
- event handling
- passing functions as arguments
- implementing strategies
- C-style APIs
- low-level programming

Modern C++ often uses **lambdas**, **functors**, and **std::function** for similar tasks.

Function as an Argument

A function can receive another function.

Example:

```
int add(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

```
void calculate(int (*operation)(int, int))
```

```
{
```

```
    cout << operation(10, 20);
```

```
}
```

Call:

```
calculate(add);
```

Output:

30

This is an important concept behind **callbacks**.

```
#include <iostream>
```

```
using namespace std;
```

```
/*
```

```
inline:
```

In a normal function call whenever a call is made, control is transferred to the function. The function executes and control is given back to the script.

-ve: This makes the process a bit slow because there is a transfer of control.

Inline function is an answer to that.

In an inline function during compilation,

a function call is replaced with the function body

within the code itself.

+ve: This maintains the speed because control transfer does not take place.

-ve: Also increases the size of the code because we are adding code to the existing object file, so the object file will be bigger.

take on it: An inline function should not be more than one line.

```
*/  
  
inline int add(int no1,int no2)  
    {  
        return (no1+no2);  
    }  
  
int main()  
{  
  
    cout<<"Addition is:"<<add(10,20);  
  
    return 0;  
}
```